ProvenanceRank

Complete Project Documentation

2026-06-28

Contents

ProvenanceRank — Documentation

A complete, beginner-friendly guide to the project: what it does, how it's built, the technology behind every feature, and how you could rebuild it from scratch.

Read the files in order if you're new. Jump straight to **04** if you just want to know what a particular file does.

> **PDF versions** are provided too: a single combined `ProvenanceRank_Documentation.pdf` (the whole thing, with a table of contents), and a per-file PDF for each document under `pdf/`. The Markdown files are the source of truth.

#	File	What's inside
00	this file	Index + how to read the docs
01	01_overview.md	The problem, what the system does, the full tech stack, repo map
02	02_architecture.md	The two-phase design, data flow, how the layers fit, diagrams
03	03_features.md	Every feature, the tech behind it, and how it works
04	04_file_by_file.md	A reference for **every** module and file in the repo
05	05_build_from_scratch.md	Step-by-step: rebuild the project yourself, in order
06	06_glossary.md	Plain-English definitions: NDCG, BM25, RRF, LambdaMART, etc.
07	07_running_guide.md	**How to run every section** — engine, tests, API, UI, live layer, graph RAG, Docker, with copy-paste commands

The 30-second version

ProvenanceRank takes a **job description** and a file of **100,000 candidate profiles** and produces a ranked **top-100** shortlist (`submission.csv`), each row with a one-line justification. It was built for a hiring-AI hackathon (rank a "Senior AI Engineer — Founding Team" role) and then grown into a production-style service with an API, database, authentication, observability, a React UI, and a "live" career-intelligence layer that ingests real GitHub activity into a verifiable proof-of-work graph.

The graded part runs under hard constraints: **no network, CPU only, under 5 minutes, exactly 100 rows.** The trick that makes that possible is a **two-phase split** — all the slow, smart work happens *offline* and is cached; the graded step just reads the cache. See 02_architecture.md.

Other docs in the repo

These pre-date this folder and go deeper on specific areas:

- `../README.md` — top-level project readme
- `../RUN_LOCAL.md` — how to run it on your laptop (3 ways)
- `../ACCURACY.md` — the accuracy + performance layers in depth
- `../SYSTEM_DESIGN.md` — production system-design write-up
- `../ARCHITECTURE.md` — architecture diagram + rationale

Conventions used in these docs

- **"Offline"** = the `precompute.py` phase. Allowed to be slow and use the network.
- **"Online"** / **"graded"** = the `rank.py` phase. No network, CPU, < 5 min.
- **"Graceful degradation"** = every heavy dependency (torch, xgboost, Redis, Neo4j, Postgres, a GPU, an API key) is *optional*. If it's missing, the code falls back to a pure-Python path and still runs. This is a core design rule, so you'll see it mentioned a lot.

01 — Overview

The problem

You're given:

- **`candidates.jsonl`** — 100,000 anonymised candidate profiles, one JSON object per line. Each has a profile (headline, summary, title, location, years of experience), a career history (roles, companies, durations, descriptions), skills (with endorsements and self-rated proficiency), behavioural signals (activity, recruiter responses, assessment scores), and verification flags.
- **A job description (JD)** — for the hackathon, a "Senior AI Engineer — Founding Team" role.

You must produce **`submission.csv`** — the **top 100** candidates ranked best-fit first, each with a `score` and a one-sentence `reasoning`.

The hard constraints (these shape every design decision)

1. **No network and no GPU during ranking.**
2. **CPU only, ≤ 16 GB RAM, under 5 minutes** for the ranking step.
3. **Exactly 100 rows**, valid against the organiser's `validate_submission.py`.
4. The dataset hides **~80 "honeypot" profiles** — deliberately impossible resumes (e.g. 40 years of experience at age 25) that you must detect and exclude.
5. The official score is a weighted blend of ranking-quality metrics:

   `score = 0.50·NDCG@10 + 0.30·NDCG@50 + 0.15·MAP + 0.05·P@10`

   (Don't worry if those are unfamiliar — they're all defined in 06_glossary.md. In short: did you put the right people near the top?)

What the system does (the whole thing)

The project grew in four layers. Each one is useful on its own and degrades gracefully if its dependencies aren't installed.

1. **The static ranker (the graded core).** Reads profiles, builds 40 numeric features per candidate, retrieves the most relevant ones with a hybrid keyword + semantic search, scores them with a machine-learning model, removes honeypots and unqualified candidates, and writes the ranked top-100.
2. **The accuracy layer.** Three techniques that sharpen the *top* of the list, where the score is decided: graded "pseudo-labels" feeding a learning-to-rank model (LambdaMART), a cross-encoder **reranker**, and a pairwise **Bradley-Terry tournament** (our own differentiator). Plus an evaluation harness that measures the exact contest metric so changes are proven, not guessed.
3. **The production service.** A FastAPI application with a real database (Postgres/SQLite), Redis caching, a job queue, authentication (JWT + API keys) with role-based access, Prometheus metrics, structured logging, distributed tracing, a circuit breaker, rate limiting, Docker packaging, CI, and a React recruiter UI.
4. **The live signal ingestion layer.** A passive career-intelligence agent that pulls a developer's real GitHub activity, turns each piece of work into evidence, scores how much that evidence is worth (with recency decay), stores it in a SHA-256-anchored "proof-of-work" knowledge graph, and lets a recruiter query it in plain English. It composes with the static ranker through a small additive "evidence bonus" — it never overrides the graded submission.
5. **The performance layer.** Speed levers that make the offline phase cheaper without weakening any feature: GPU auto-detection, an ONNX/int8 toggle, leaner model training, a focused rerank text, and a content-hash cache for re-runs.

The full technology stack

Core ranking engine (pure-Python-capable)

Concern	Technology	Notes
Numerics / data	**NumPy**, **pandas**	The feature matrix is a pandas DataFrame
Keyword search	**rank-bm25** (Okapi BM25)	Falls back to a hand-written streaming inverted index
Semantic search	**sentence-transformers** (MiniLM, 384-dim)	Falls back to a NumPy feature-hashing embedder
Fusion	**Reciprocal Rank Fusion** (custom)	Combines keyword + semantic ranks
ML model	**XGBoost** (`XGBRanker`, LambdaMART)	Falls back to LightGBM → scikit-learn → a formula
Reranker	**sentence-transformers** `CrossEncoder`	Falls back to a lexical BM25-style scorer
Tournament	**NumPy** (Bradley-Terry MLE)	`scipy` listed but the core is pure NumPy
Model I/O	**joblib**	Saves/loads the trained model artifact
LLM (optional, offline)	**google-generativeai** (Gemini)	JD parsing + label grading; heuristic fallback

Production service

Concern	Technology
Web framework	**FastAPI** + **uvicorn** / **gunicorn**
Streaming progress	**sse-starlette** (Server-Sent Events)
Database (async)	**SQLAlchemy 2.0** + **asyncpg** (Postgres) / **aiosqlite** (dev)
Migrations	**Alembic**
Cache + rate limit	**Redis** (with an in-memory fallback)
Auth	**PyJWT** (JWT) + **bcrypt** (passwords) + per-user API keys
Validation	**Pydantic v2** + **pydantic-settings**
Metrics	**prometheus-client**
Logging	**structlog** (JSON to stdout)
Tracing	**OpenTelemetry** (optional, exports to Jaeger)
Background work	**Celery** (eager fallback = runs inline)
Knowledge graph	**Neo4j** (with an in-memory fallback)
Resume parsing	**pdfplumber** / **python-docx** (optional)

Frontend

Concern	Technology
Framework	**React 18** + **TypeScript**
Build tool	**Vite**
Styling	**Tailwind CSS**
Data fetching	**TanStack React Query**
Routing	**react-router-dom**
Charts	**Recharts**
Icons	**lucide-react**

Infrastructure & quality

Concern	Technology
Containers	**Docker** multi-stage + **docker-compose** (13 services)
Reverse proxy / LB	**nginx** (the `gateway` service)
Dashboards	**Grafana** + **Prometheus**
Tracing UI	**Jaeger**
CI	**GitHub Actions**
Lint / format	**Ruff**

Concern	Technology
Tests	**pytest** + **pytest-asyncio** (68 tests)

Repository map (top level)

```
provenancerank/
├── precompute.py          # PHASE 1 (offline): build all cached artifacts
├── rank.py                # PHASE 2 (online): read cache -> submission.csv
├── validate_submission.py # organiser's official validator (do not edit)
│
├── core/          # config, logging, constants, security, device, cache, errors
├── pipeline/      # the ranking engine: features, retrieval, scoring, honeypots
├── ml/            # model training, prediction, pseudo-labels, reranker, metrics
├── output/        # reasoning generator + submission writer
│
├── api/           # FastAPI app: routers, schemas, deps, middleware, metrics
├── db/            # SQLAlchemy models, repositories, engine, migrations
├── services/      # ranker engine wrapper, job queue, cache, rate limiter
│
├── ingestion/     # GitHub client, resume parser, sync core, Celery tasks
├── intelligence/  # confidence scorer, skill extractor, artifact summariser
├── graph/         # knowledge-graph schema, read/write, natural-language query
│
├── frontend/      # React + TypeScript recruiter UI
├── alembic/       # database migration scripts
├── deploy/        # Prometheus + Grafana provisioning
├── scripts/       # dev helpers (e.g. build a small sample dataset)
├── tests/         # pytest suite (63 tests)
│
├── docker-compose.yml, Dockerfile, .github/workflows/ci.yml
└── docs/          # you are here
```

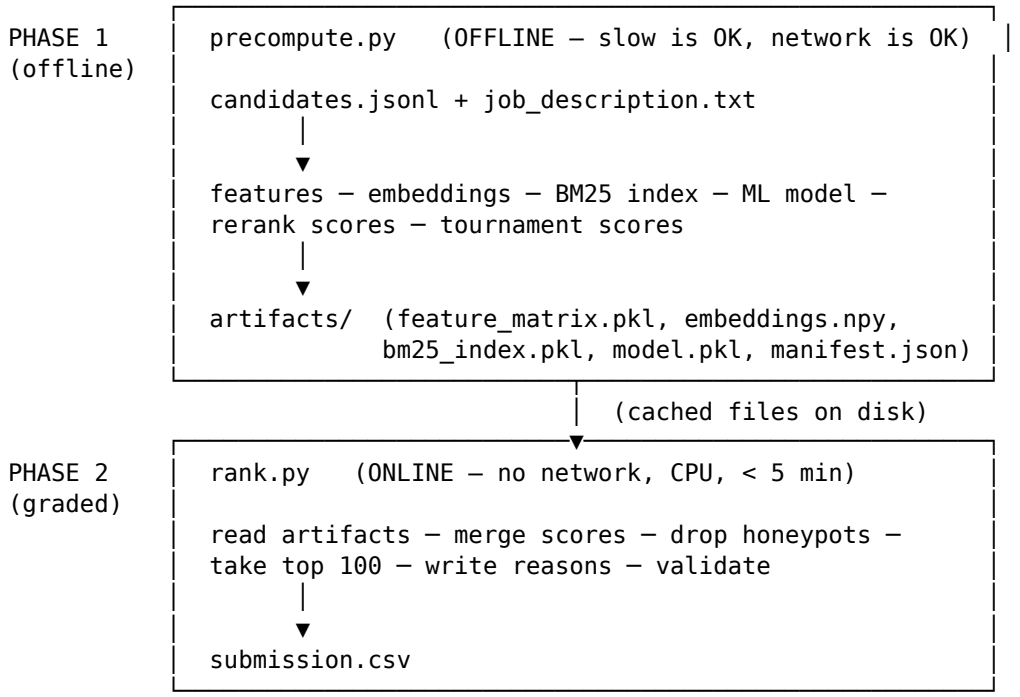
Every one of these files is described in 04_file_by_file.md. The next doc, 02_architecture.md, explains how they fit together.

02 — Architecture

The one idea that makes everything work: two phases

The graded ranking step has to run with **no network, on CPU, in under 5 minutes**. But the *smart* parts of ranking — embedding 100,000 profiles with a neural model, training a gradient-boosted ranker, running a cross-encoder — are slow and sometimes need the network. You can't do both in one step.

So the system is split in two:



Everything expensive happens in **Phase 1** and is written to the `artifacts/` folder. **Phase 2** only *reads* those files and does cheap arithmetic, so it comfortably meets the constraints. This is why adding more "intelligence" (a reranker, a tournament, GitHub evidence) never slows down the graded step — it all lands in Phase 1.

Phase 1 in detail (`precompute.py`)

```
1. Hash inputs ──► if --resume and nothing changed, skip everything.
2. Deconstruct the JD ──► a JobSpec (required skills, seniority, location).
                          (the only place an LLM may run; falls back to rules)
3. Stream candidates once ──► build:
      • the 40-feature row per candidate   (pipeline/feature_engineering.py)
      • the full text doc (for search)      (pipeline/loader.corpus_text)
      • a focused text doc (for rerank)     (pipeline/loader.rerank_text)
4. Apply hard gates ──► mark honeypots / unqualified (still kept in the matrix,
                        flagged, so the API can show *why* they were excluded).
5. Embeddings ──► encode all docs (sentence-transformers or hashing). Cached.
6. BM25 index ──► build keyword index. Score the JD query against it.
7. Retrieval ──► fuse BM25 + embedding cosine with Reciprocal Rank Fusion.
8. Accuracy layer:
      • graded pseudo-labels ──► train the LambdaMART ranker (saved to model.pkl)
      • cross-encoder rerank of the top ~300 ──► rerank_score column
      • Bradley-Terry tournament of the top ~60 ──► tournament_score column
9. Persist ──► feature_matrix.pkl, embeddings.npy, bm25_index.pkl, model.pkl,
               manifest.json, candidate_ids.txt.
```

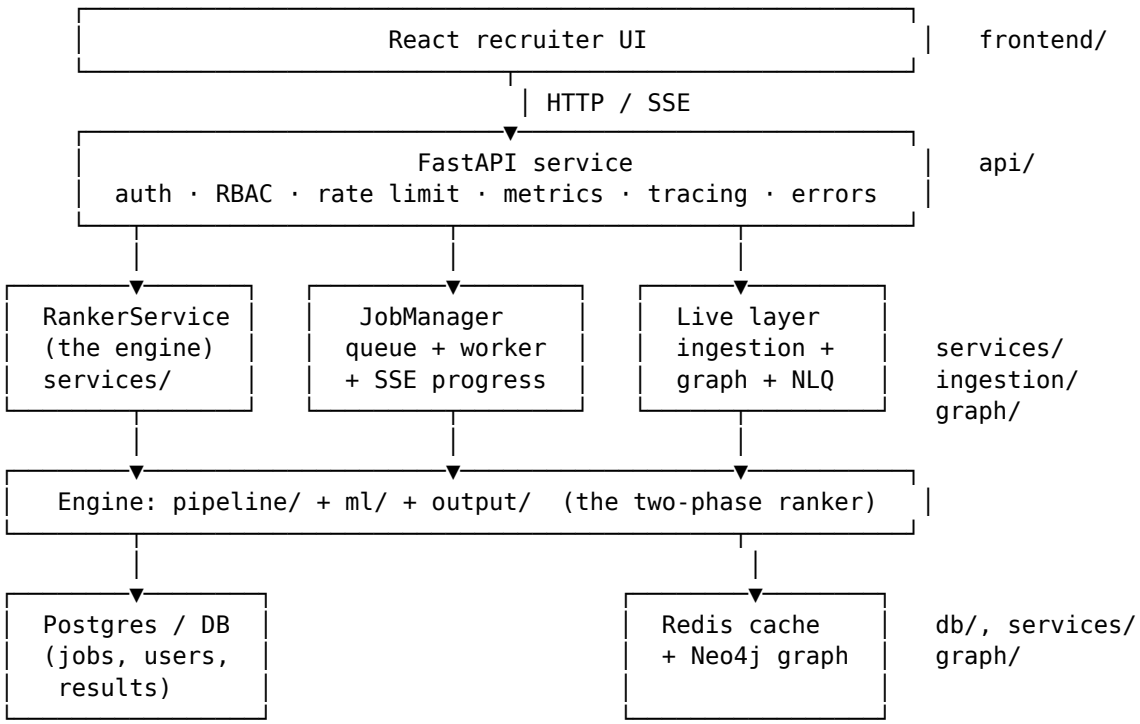
Phase 2 in detail (`rank.py`)

```
1. Load artifacts (feature_matrix.pkl, the model, embedding meta).
2. predict_fit() ──► the ML "fit" score per candidate (loads model.pkl).
3. merge_final_scores() ──► combine into one number:

     core   = 0.40·ml_fit + 0.35·retrieval + 0.25·behavioural
     base   = (1 − w_head)·core + w_rerank·rerank + w_tour·tournament   ← head only
     final  = base · location_factor · signal_modifier + evidence_bonus

4. Force honeypots and gated candidates to score 0 (they can never reach top 100).
5. Sort, take the top 100.
6. reasoning_generator ──► a one-line human justification per pick.
7. submission_writer ──► write submission.csv and run the official validator.
```

There are actually two ways into Phase 2:

- **Fast path** — the requested candidates are already in the precomputed matrix, so it just reads cached features. This is the graded path (milliseconds).
- **Live path** — brand-new candidates not seen at precompute time (e.g. an API caller submits fresh profiles). It computes their features on the fly. Slower, but flexible. The fast path falls back to this automatically.

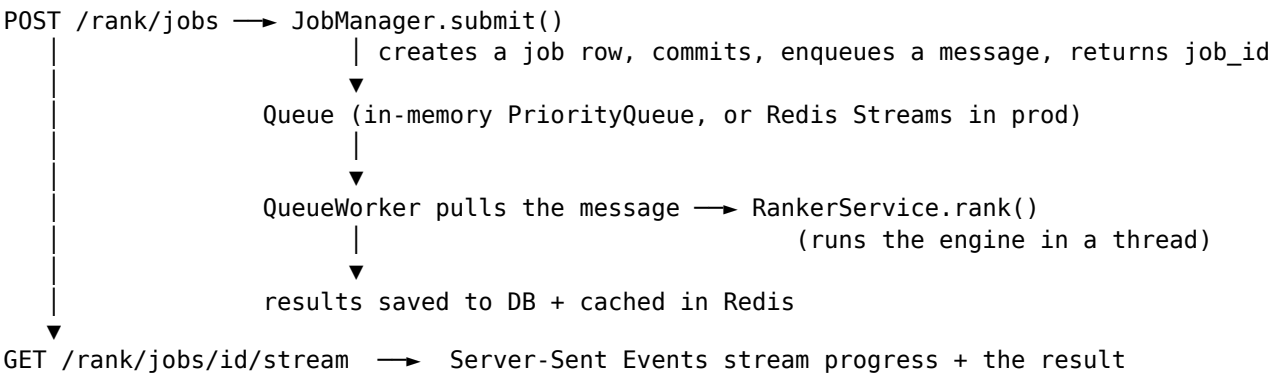
How the four layers stack



Crucially, the **engine doesn't depend on the service.** `precompute.py` and `rank.py` run from the command line with nothing else installed. The API, DB, cache, and live layer are wrappers *around* the engine, so a failure in any of them can't break the graded submission. This is "microservice-style isolation" applied within one codebase: each concern is a separate module with a fallback.

Data flow for a single ranking request (production)

```
Recruiter clicks "Run ranking"
   │
   ▼
```



The worker is **idempotent** (a duplicate message for a finished job is a no-op) and **at-least-once** (failures retry, then dead-letter). At scale, the worker moves to its own machine fleet without changing the API — that's the whole point of putting a queue in the middle.

The graceful-degradation ladder

Every external dependency is optional. The system picks the best available option and silently drops to the next:

Capability	Best	→	→	Floor
Embeddings	sentence-transformers (GPU)	(CPU)	—	NumPy hashing embedder
Keyword search	rank-bm25	—	—	hand-written inverted index
ML ranker	XGBoost LambdaMART	LightGBM	scikit-learn	linear formula
Reranker	CrossEncoder (GPU/ONNX)	(CPU)	—	lexical BM25-style scorer
JD parsing / labels	Gemini LLM	—	—	rule-based heuristics
Database	Postgres (asyncpg)	—	—	SQLite (aiosqlite)
Cache	Redis	—	—	in-memory LRU
Knowledge graph	Neo4j	—	—	in-memory graph store
Background jobs	Celery + Redis	—	—	run inline (eager)

This is why the project runs end-to-end in a bare sandbox with only NumPy and pandas, *and* scales up to a full GPU + Postgres + Redis + Neo4j deployment — same code, different rungs of the ladder.

Continue to 03_features.md for each feature and the tech behind it, or jump to 04_file_by_file.md for the module reference.

03 — Features (and the technology behind each)

Every feature below lists **what it does**, the **tech** it uses, **how it works**, and its **fallback** (the pure-Python path used when a heavy dependency is missing). File references point you to 04_file_by_file.md.

A. The ranking engine

A1. Streaming candidate loader

- **What:** reads the 100K-line `candidates.jsonl` without loading it all into RAM.
- **Tech:** Python generators, `json`, `hashlib` (SHA-256 of the file for cache keys).
- **How:** `iter_candidates()` yields one parsed profile at a time; `corpus_text()` flattens a profile into one searchable string; `rerank_text()` builds a shorter, focused string (title + recent role + top skills) for the cross-encoder.
- **Files:** `pipeline/loader.py`.

A2. Feature engineering — 40 numeric features

- **What:** turns each messy profile into a fixed vector of 40 numbers — the single contract every model in the system agrees on.
- **Tech:** pandas, NumPy; date math for tenure/recency.
- **How:** `build_feature_row()` computes things like years of experience, average tenure, JD-skill match count, retrieval-skill count, assessment scores, product-company ratio, behavioural composite, and location fit. The exact list lives in `core/constants.NUMERIC_FEATURE_COLUMNS`. Because it's a fixed schema, you can swap the model behind it without touching anything else.
- **Files:** `pipeline/feature_engineering.py`, `core/constants.py`.

A3. Honeypot detection

- **What:** catches the ~80 fake "trap" profiles (impossible career arithmetic).
- **Tech:** rule-based, vectorised pandas — chosen deliberately over ML for **high precision** (we must not flag real people).
- **How:** a few strict rules (e.g. total role durations exceed the person's age, or experience that can't fit a plausible timeline). Honeypots are *kept* in the matrix but flagged, so the API can explain the exclusion; the scorer pins their score to 0 so they can never enter the top 100. On the real data this flags ~65.
- **Files:** `pipeline/honeypot_detector.py`.

A4. JD deconstruction

- **What:** turns the free-text job description into a structured `JobSpec` (required skills, seniority, location preference, must-haves).
- **Tech:** **Gemini** (`google-generativeai`) *optionally*, offline only.
- **How:** if an API key is set, an LLM extracts structured requirements; otherwise a keyword/rule parser does it. This is the **only** place an LLM may run, and only in Phase 1 — so the graded Phase 2 never needs the network.
- **Files:** `pipeline/jd_deconstructor.py`.

A5. Gate filter

- **What:** removes candidates who fail hard requirements before ranking.
- **Tech:** vectorised pandas; no LLM, no network.

- **How:** `apply_gates()` checks the `JobSpec` requirements against the feature matrix (e.g. "all-consulting career", keyword-stuffer, inactive/unresponsive). Gated rows are flagged and written to `excluded_candidates.csv` with a reason.
- **Files:** `pipeline/gate_filter.py`.

A6. Semantic embeddings (dense retrieval)

- **What:** represents each profile and the JD as a 384-dimension vector so that *meaning*, not just words, can be compared.
- **Tech: sentence-transformers** (`all-MiniLM-L6-v2`).
- **How:** `embed_corpus()` encodes every profile; the JD is encoded the same way; cosine similarity ranks profiles by semantic closeness to the JD.
- **Fallback:** a **NumPy feature-hashing embedder** — hashes tokens into buckets with signs and L2-normalises. Deterministic, offline, no torch. Coarser, but the rest of the pipeline can't tell the difference (same 384 dims).
- **Files:** `pipeline/embedder.py`.

A7. BM25 keyword index (sparse retrieval)

- **What:** classic keyword relevance — which profiles literally contain the JD's important terms, weighted by rarity.
- **Tech: rank-bm25** (Okapi BM25).
- **How:** builds an index over all profile texts, scores the JD query against it.
- **Fallback:** a hand-written **two-pass streaming inverted index** with document-frequency pruning — built to keep memory low on 100K docs (this is what fixed an out-of-memory problem; see 04).
- **Files:** `pipeline/bm25_indexer.py`.

A8. Hybrid retrieval (Reciprocal Rank Fusion)

- **What:** combines keyword (BM25) and semantic (embedding) rankings into one.
- **Tech: Reciprocal Rank Fusion (RRF)** — a simple, robust rank-combining formula: `score = Σ 1/(k + rank_in_each_list)`.
- **How:** each method ranks the candidates; RRF blends the *ranks* (not the raw scores, which are on different scales). The result is the `retrieval_score`.
- **Files:** `pipeline/retrieval.py`.

A9. ML fit scorer

- **What:** a machine-learning model that predicts how well a candidate fits.
- **Tech: XGBoost** (`XGBRanker`, an NDCG-objective LambdaMART). Falls back to **LightGBM**, then **scikit-learn** `HistGradientBoostingRegressor`, then a hand-tuned linear **formula**.
- **How:** the model is trained in Phase 1 (see A11) and saved with **joblib**; `predict_fit()` loads it and produces a normalised `ml_fit` score per candidate. If the model file is missing or won't load, it returns the formula directly — the ranker still works.
- **Files:** `ml/trainer.py`, `ml/predictor.py`.

B. The accuracy layer (sharpening the top of the list)

B1. Graded pseudo-labels → LambdaMART

- **What:** the dataset has no "hire/reject" labels, so we *manufacture* graded relevance labels (tier 0–5) and train a learning-to-rank model to optimise the contest metric directly.
- **Tech:** percentile bucketing (pandas/NumPy); optionally **Gemini** to grade a sample; **XGBRanker** with `objective="rank:ndcg"` (LambdaMART).

- **How:** a richer-than-proxy composite is bucketed into tiers (a few "5"s, more "3"s, lots of "0"s — like real relevance). If a key is set, Gemini grades the top ~1,500 against the JD and overrides the heuristic for that slice. The ranker then learns the *actual* definition of relevance, not a proxy.
- **Fallback:** heuristic tiers (no LLM) and the regressor/formula (no XGBoost).
- **Files:** `ml/pseudo_labels.py`, `ml/trainer.py`.

B2. Cross-encoder reranker

- **What:** re-scores the *top ~300* candidates by reading the JD and a profile **together** in one transformer pass — far more precise than scoring them apart.
- **Tech:** **sentence-transformers** `CrossEncoder` (`ms-marco-MiniLM-L-6-v2`).
- **How:** only the head is reranked (that's where NDCG@10/@50 is decided); reranking all 100K would be too slow. The result is cached as `rerank_score`.
- **Fallback:** a **lexical** BM25-flavoured overlap of JD terms with each profile — pure NumPy, still a real signal.
- **Files:** `ml/reranker.py`.

B3. Bradley-Terry tournament (our differentiator)

- **What:** for the very top (~60), runs an actual **round-robin tournament** — "between these two, who's better?" — and resolves it into a globally consistent ranking.
- **Tech:** **NumPy**; Bradley-Terry maximum-likelihood via MM iteration (Hunter, 2004).
- **How:** every candidate "plays" every other on several JD-relevant criteria; each match is a soft win-share; the round-robin (which can contain cycles like A>B>C>A) is resolved by the Bradley-Terry model into a single strength per candidate. Because only *ordinal* comparisons matter, it's immune to feature-scale quirks. Cached as `tournament_score`.
- **Files:** `pipeline/tournament.py`.

B4. LLM head-scoring (the smartest signal)

- **What:** an LLM reads each *top ~250* candidate against the JD and scores fit 0–100 — judgment the GBDT can't do (it reads the career narrative, not just numbers).
- **Tech:** GLM / Ollama / Gemini via `core/llm.py`; runs **offline in precompute**, batched (~20 candidates per call), cached as `llm_fit_score`.
- **How:** only the head is scored (where NDCG@10/@50 is decided); the result is blended into the head like the rerank/tournament signals. `rank.py` reads the cached column, so it stays no-network/<5min.
- **Fallback:** off by default (`LLM_HEAD_SCORING_ENABLED=true` to enable). With no LLM reachable it scores nothing and the scorer adds nothing — baseline unchanged.
- **Files:** `ml/llm_scorer.py`, `pipeline/scorer.py`, `precompute.py`.

B5. Evaluation harness

- **What:** computes the exact contest metric so every change is measured.
- **Tech:** pure NumPy.
- **How:** implements `NDCG@k`, `MAP`, `P@k` and the weighted `composite`. We score against the graded pseudo-labels (a stand-in for the hidden ground truth) to A/B any change and confirm the head actually improved.
- **Files:** `ml/evaluate.py`.

B6. Final score merge

- **What:** combines every signal into one number per candidate.
- **Tech:** pandas/NumPy.
- **How:** `core = 0.40·ml_fit + 0.35·retrieval + 0.25·behavioural`; the rerank and tournament signals fold into the head as a **convex blend** (so scores stay in [0,1] and don't saturate);

then × `location_factor` × `signal_modifier` + `evidence_bonus`. Honeypots/gated → 0. When the extra columns are absent, the formula is exactly the original — so the add-ons are invisible by default.
- **Files:** `pipeline/scorer.py`, `pipeline/signal_scorer.py`.

C. Output

C1. Reasoning generator

- **What:** the one-sentence human justification on each shortlisted candidate.
- **Tech:** template-based prose (no LLM needed at rank time).
- **How:** picks the most salient, JD-relevant facts (seniority, proven skills, product-company experience, activity) and writes a varied, readable sentence.
- **Files:** `output/reasoning_generator.py`.

C2. Submission writer + validator

- **What:** writes `submission.csv` and checks it against the official rules.
- **Tech:** pandas; the organiser's `validate_submission.py`.
- **How:** assembles exactly 100 rows (`candidate_id, rank, score, reasoning`), writes the CSV, then runs the validator to guarantee it passes before you submit.
- **Files:** `output/submission_writer.py`, `validate_submission.py`.

C3. Coverage loop agent

- **What:** a lightweight "are we missing anything?" check over the shortlist.
- **Tech:** pure Python (no LLM, no network).
- **How:** a plan→retrieve→reflect loop reports coverage across dimensions (retrieval, LLM experience, production evidence, availability). It's informational — it does not change the final order.
- **Files:** `pipeline/loop_agent.py`.

D. The production service

D1. FastAPI application

- **What:** the HTTP API around the engine.
- **Tech:** **FastAPI**, **uvicorn**/**gunicorn**, **Pydantic v2**, **sse-starlette**.
- **How:** an app factory wires routers (auth, rank, developers, search, admin, health), middleware, error handlers, metrics and tracing. Validation happens at the edge with Pydantic schemas.
- **Files:** `api/` (app.py, main.py, deps.py, schemas.py, routers/*).

D2. Authentication, API keys & RBAC

- **What:** logins, machine keys, and role-based permissions.
- **Tech:** **PyJWT** (JWT access/refresh tokens), **bcrypt** (password hashing), per-user API keys.
- **How:** password login issues JWTs; users mint API keys (shown once, stored hashed). A dependency authenticates either a key *or* a JWT and enforces roles (admin / recruiter / viewer).
- **Files:** `core/security.py`, `api/deps.py`, `api/routers/auth.py`.

D3. Database & repositories

- **What:** persistent storage for users, jobs, results, audit logs, and the live layer's developers/evidence.

- **Tech: SQLAlchemy 2.0 async** + **asyncpg** (Postgres) / **aiosqlite** (dev); **Alembic** migrations.
- **How:** ORM models use UUID string PKs and portable JSON columns so the *same* schema runs on Postgres and SQLite. All DB access goes through a **repository layer**, so routers/services never write raw queries.
- **Files:** `db/models.py`, `db/repositories.py`, `db/base.py`, `alembic/`.

D4. Caching & rate limiting

- **What:** caches expensive results; throttles abusive callers.
- **Tech: Redis** with an in-memory fallback.
- **How:** a Cache abstraction supports read-through caching and eviction (LRU/ LFU). A fixed-window rate limiter uses Redis `INCR` (or in-memory counters).
- **Files:** `services/cache.py`, `services/ratelimit.py`.

D5. Job queue & orchestration

- **What:** ranking runs as a background job; the request returns a `job_id` immediately.
- **Tech:** an in-process **asyncio PriorityQueue** (dev) or **Redis Streams** with consumer groups (prod); **SSE** for live progress.
- **How:** `JobManager.submit()` records the job, commits, enqueues a message, and returns. A `QueueWorker` runs the engine in a thread. Delivery is *at-least-once* with retries and a dead-letter queue; the consumer is *idempotent*.
- **Files:** `services/jobs.py`, `services/queue.py`, `services/ranker.py`.

D6. Observability & resilience

- **What:** metrics, logs, traces, and failure isolation.
- **Tech: prometheus-client**, **structlog** (JSON), **OpenTelemetry** (→ Jaeger), a custom async **circuit breaker**.
- **How:** middleware records request metrics and a request-id on every log line; spans trace requests when tracing is enabled; the circuit breaker stops hammering a failing dependency (e.g. the GitHub API) and recovers automatically.
- **Files:** `api/metrics.py`, `core/logging.py`, `core/tracing.py`, `core/circuit_breaker.py`, `api/middleware.py`.

E. The live signal ingestion layer

E1. GitHub ingestion

- **What:** pulls a developer's real work (commits, PR reviews, issues) from GitHub.
- **Tech: GitHub GraphQL API**, async **httpx**, the circuit breaker.
- **How:** an async client fetches repos and contributions within rate limits; the sync core is shared by the API (await directly) and Celery (background).
- **Fallback:** without a token, ingestion simply no-ops.
- **Files:** `ingestion/github_client.py`, `ingestion/sync.py`, `ingestion/tasks.py`.

E2. Evidence confidence scoring

- **What:** how much to trust a skill claim, based on the evidence behind it.
- **Tech:** a transparent weighted formula (NumPy/Python).
- **How:** `confidence = clamp(Σ source_weight · (complexity/5) · 2^(−age/365) / normaliser, 0, 1)`. A merged PR (`commit_diff`, weight 1.0) is worth more than a resume claim (0.2); evidence **decays** with a one-year half-life. So fresh, hard evidence outranks stale, soft claims.
- **Files:** `intelligence/confidence_scorer.py`.

E3. Skill extraction & artifact summaries

- **What:** map messy skill strings to canonical names; summarise each work item.
- **Tech: spaCy** (optional) for skill extraction; **Gemini** (optional) for summaries.
- **How:** skills are normalised to a taxonomy; each artifact (commit, PR, issue) gets a short natural-language summary. Both fall back to deterministic heuristics with no model.
- **Files:** `intelligence/skill_extractor.py`, `intelligence/summariser.py`.

E4. Proof-of-work knowledge graph + hybrid (graph + vector) query

- **What:** stores evidence in a graph and lets a recruiter query it in English, fusing exact graph traversal with semantic search for better recall.
- **Tech: Neo4j** (with an in-memory graph fallback); SHA-256 anchoring; embeddings (sentence-transformers or the NumPy hashing fallback); RRF fusion.
- **How:** developers → skills → evidence are nodes/edges; each evidence node is content-hashed so it's *verifiable* (tamper-evident). A query runs **two** retrievers and fuses them by rank (RRF, like the main ranker):
  1. **structured skill traversal** — precise: developers tagged with the canonical skill (e.g. the 3-hop "production debuggers" query);
  2. **semantic vector search** over evidence summaries — recall: catches paraphrases the skill tag misses ("real-time event streaming" → a "Kafka" developer whose commit says "streaming telemetry consumer"). Confidence **accumulates** across evidence; each result is tagged `match_via` (skill / semantic / both).
- **Measured:** on a gold-set A/B, hybrid lifts recall@5 from 0.6 → 1.0 and MRR 0.6 → 1.0 vs skill-only (it answers the paraphrased queries that returned nothing before), trading some precision@k — the usual recall/precision balance.
- **Why not "full Graph RAG":** the LLM is used to *parse* the query and to *summarise* evidence at ingestion, not to generate the final answer at query time — so this is graph-RAG-style retrieval, not generative Graph RAG.
- **Files:** `graph/schema.py`, `graph/queries.py`, `graph/vector_index.py`, `graph/natural_language_query.`
  `graph/evaluate.py`.

E5. Composition with the static ranker

- **What:** verified developers float slightly above identical static-only twins.
- **How:** the graph contributes an additive `evidence_confidence_bonus` column that the scorer adds at the end. It defaults to 0, so the graded submission is **unchanged** when the live layer is off. It augments, never overrides.
- **Files:** `pipeline/feature_engineering.py` (apply_live_signals), `pipeline/scorer.py`.

F. Frontend (recruiter UI)

- **What:** a web console to run rankings, watch progress, browse results, and search the evidence graph.
- **Tech: React 18 + TypeScript**, **Vite**, **Tailwind CSS**, **TanStack React Query**, **react-router-dom**, **Recharts**, **lucide-react**.
- **How:** pages for Login, Dashboard, Ranking Results, Natural-Language Search, and Settings; a Server-Sent-Events hook streams job progress; an evidence timeline and proof-of-work card visualise the live layer.
- **Files:** `frontend/src/**`.

G. Performance layer (speed, no feature loss)

- **Device auto-detect** — runs the transformers on Apple **MPS** / **CUDA** when available, else CPU. `core/device.py`.
- **ONNX + int8 toggle** — optional ONNX Runtime backend with quantised weights (~2–4× on CPU), with onnx→torch fallback. `core/device.py`.
- **Leaner LambdaMART** — trains on all positives + a sampled set of negatives, histogram tree method, early stopping. `ml/trainer.py`.
- **Focused rerank text** — the cross-encoder reads a tight projection, not the full blob. `pipeline/loader.rerank_text`.
- **Content-hash cache** — `--resume` reuses embeddings / model / rerank scores when their inputs are unchanged; change only the JD and the 100K embeddings are reused. `core/artifact_cache.py`.

Full detail in `../ACCURACY.md`. Next: 04_file_by_file.md.

04 — File-by-file reference

Every file in the repo, grouped by folder, with its purpose and the key things inside it. Line counts are approximate and just give a sense of size. Generated artifacts, `node_modules`, and the dataset are omitted.

Entry points (repo root)

File	~Lines	Purpose
`precompute.py`	322	**Phase 1 (offline).** Builds every artifact `rank.py` reads: features, embeddings, BM25 index, the trained model, rerank + tournament scores. Has the content-hash cache and `--resume`, `--no-st`, `--no-accuracy`, `--limit`, `--bm25-backend` flags.
`rank.py`	282	**Phase 2 (online).** No network, CPU, < 5 min. Loads artifacts, predicts fit, merges scores, drops honeypots, takes top 100, writes reasons, validates. Has a fast path (cached features) and a live path (compute features on the fly).
`validate_submission.py`	166	The **organiser's official validator** — checks the CSV against the challenge rules. Treated as read-only; we run it but don't edit it.

`core/` — cross-cutting foundations

File	~Lines	Purpose
`config.py`	299	**Every tunable in one place**, read from environment / `.env`. Uses pydantic-settings if available, else a plain-dataclass fallback. Holds weights, paths, model names, device/ONNX flags, training knobs, DB/Redis/Neo4j URLs, JWT settings, etc.
`constants.py`	373	**Single source of truth** for domain vocabularies (skill taxonomies: retrieval, LLM, core-ML, CV/speech/robotics; consulting firms) and `NUMERIC_FEATURE_COLUMNS` — the 40-feature ML contract.
`logging.py`	151	**Structured JSON logging** to stdout via structlog, plus a `log_duration` context manager that times a block and attaches metrics.
`security.py`	116	**Passwords (bcrypt), JWTs (PyJWT), and API keys.** Hash/verify passwords, mint/decode tokens, generate+hash API keys.
`device.py`	96	**Performance.** Picks the best compute device (MPS/CUDA/CPU) and builds the optimised→plain constructor attempts for the transformers (ONNX-int8 → ONNX → torch). Never raises.

File	~Lines	Purpose
`llm.py`	175	**One LLM call for the whole project.** `complete()` / `complete_json()` try **GLM** (Z.ai, OpenAI-compatible) → local **Ollama** → **Gemini** → None (callers use heuristics). Used by JD parse, summaries, label grading, NL query.
`artifact_cache.py`	89	**Performance.** Content-addressed cache: each stage stores a `<file>.key` sidecar; a re-run reuses the artifact only if the recomputed key matches. A stale/unreadable cache is a clean miss, never a wrong result.
`circuit_breaker.py`	75	A small **async circuit breaker** — opens after repeated failures (e.g. GitHub down), short-circuits calls, half-opens to test recovery.
`exceptions.py`	95	**Domain exceptions**, each carrying a stable machine code and an HTTP status, so the API can translate them into clean error responses.
`tracing.py`	91	**OpenTelemetry** setup (optional). Configures spans/exporter to Jaeger when enabled; a no-op otherwise.
`celery_app.py`	53	The **Celery app** for ingestion/intelligence workers + the beat scheduler. Defaults to “eager” (runs inline) so no broker is needed in dev.

`pipeline/` — the ranking engine

File	~Lines	Purpose
`loader.py`	136	**Streaming loader.** `iter_candidates()` (one profile at a time), `corpus_text()` (full searchable doc), `rerank_text()` (focused doc for the cross-encoder), `file_sha256()`, `count_lines()`.
`feature_engineering.py`	486	**The 40-feature builder.** `build_feature_row()` + `frame_from_rows()`. Also `apply_live_signals()` (adds the evidence-bonus columns, default 0) and skill-confidence helpers. The biggest, most central pipeline file.
`honeypot_detector.py`	106	**High-precision rule-based** detection of the ~80 fake profiles (impossible career arithmetic). Tuned to avoid false positives.
`jd_deconstructor.py`	297	**JD → JobSpec.** The only place an LLM (Gemini) may run, offline only; rich rule-based fallback. Saves `jd_spec.json`.
`gate_filter.py`	91	**Hard gates** at ranking time (vectorised): consulting-only careers, keyword stuffers, inactive/unresponsive. Writes the rejection registry.

File	~Lines	Purpose
`embedder.py`	157	**Semantic embeddings.** `SentenceTransformerEmbedder` (with device/ONNX) and the NumPy `HashingEmbedder` fallback; `embed_corpus()`, `make_embedder()`, meta save/load for offline rebuild.
`bm25_indexer.py`	144	**BM25 keyword index.** rank-bm25 backend + a memory-frugal streaming **inverted-index** fallback with DF pruning. `build_index()`, `query_scores()`, save/load.
`retrieval.py`	68	**Hybrid retrieval.** `cosine_scores()` (dense) + `fuse_rrf()` (Reciprocal Rank Fusion of BM25 and embeddings).
`scorer.py`	91	**Final-score merge** — the one formula, shared by `precompute` and `rank`. Convex head-blend of rerank/tournament, location & signal multipliers, evidence bonus, honeypot/gate pinning.
`signal_scorer.py`	46	**Behavioural sub-score** (availability, engagement, trust, GitHub) and the `signal_modifier` down-weight, vectorised over the matrix.
`tournament.py`	126	**Our algorithm.** Pairwise round-robin over the head, resolved by a Bradley-Terry MLE (Hunter's MM iteration). Pure NumPy. `run_tournament()`.
`loop_agent.py`	128	**Coverage loop** (plan→retrieve→reflect), pure Python. Reports shortlist coverage; informational, doesn't reorder.

`ml/` — models, labels, metrics

File	~Lines	Purpose
`trainer.py`	304	**Training.** `create_proxy_labels()`, `train_fit_scorer()` (regressor, Strategy pattern over XGBoost/LightGBM/sklearn/formula), and `train_ranker()` (LambdaMART on graded labels) with the downsample + histogram + early-stopping speed levers.
`predictor.py`	53	**Prediction.** `predict_fit()` loads the model artifact and returns normalised fit scores; falls back to the proxy formula if the model is missing/unloadable.
`pseudo_labels.py`	129	**Graded 0–5 labels** for learning-to-rank. Percentile tiering (`_bucket_to_tiers`), optional Gemini grading of a sample, heuristic otherwise.
`reranker.py`	152	**Cross-encoder reranker** of the head (sentence-transformers `CrossEncoder`), with a pure-NumPy lexical fallback. Device/ONNX aware.

File	~Lines	Purpose
`llm_scorer.py`	80	**LLM head-scoring** — `score_head()` has the LLM (GLM/Ollama/Gemini) read the top ~250 vs the JD and score fit 0–100, batched + cached as `llm_fit_score`. Offline; opt-in.
`evaluate.py`	78	**Eval harness** — `ndcg_at_k`, `average_precision`, `precision_at_k`, the weighted `composite`, plus `compare()` and `lift()` for A/B.

`output/` — results

File	~Lines	Purpose
`reasoning_generator.py`	112	Turns a feature row into a varied, JD-relevant **one-line justification** (template-based, no LLM at rank time).
`submission_writer.py`	87	Assembles exactly 100 rows, writes `submission.csv`, and runs the **official validator** on it.

`api/` — the FastAPI service

File	~Lines	Purpose
`app.py`	144	**App factory** — lifespan (startup/shutdown), middleware, error handlers, router registration, metrics/tracing wiring.
`main.py`	25	Uvicorn entry point: `uvicorn api.main:app`.
`deps.py`	113	**Dependencies** — singletons from `app.state`, auth (API key *or* JWT), RBAC role checks, DB session.
`schemas.py`	214	**Pydantic request/response models** — validation at the edge.
`middleware.py`	52	Per-request **request-id**, structured access logging, metrics timing.
`metrics.py`	42	**Prometheus** counters/histograms; the `/metrics` exposition.
`errors.py`	48	Exception handlers → clean JSON error bodies.
`routers/auth.py`	119	Login → JWTs; create/list/revoke **API keys** (shown once).
`routers/rank.py`	152	**Ranking endpoints** — submit a job, poll status, stream progress (SSE), get results.
`routers/developers.py`	132	**Live layer** — connect a GitHub account, watch the sync, read the proof-of-work profile.
`routers/search.py`	43	**Natural-language search** over the evidence graph.
`routers/admin.py`	85	Artifact status, trigger precompute, inspect audit logs/jobs.
`routers/health.py`	63	Liveness (process up) + readiness (artifacts/DB ready).

---

db/ — persistence

File	~Lines	Purpose
models.py	212	**ORM models** (users, API keys, jobs, results, audit, developers, evidence). UUID string PKs + portable JSON columns so the same schema runs on Postgres and SQLite.
repositories.py	348	**Repository layer** — all DB access. Routers/services never write raw SQL.
base.py	110	**Async engine + session** factory; SQLite WAL/pragma tuning; session_scope().
init.sql	—	Initial SQL for the Postgres container.
alembic/env.py, alembic/versions/*	57 + 135	**Alembic** migration environment + the initial schema migration.

---

services/ — engine wrappers & infra

File	~Lines	Purpose
ranker.py	284	**RankerService** — loads the engine once and keeps it warm; the rank() method the API/worker call.
jobs.py	246	**JobManager** — submit (commit-then-enqueue), the idempotent worker handler, persistence, SSE progress channels.
queue.py	247	**Message queue** — in-memory PriorityQueue + QueueWorker, and a RedisStreamQueue (consumer groups, dead-letter) with the same interface.
cache.py	148	**Cache abstraction** — Redis backend + in-memory fallback; eviction (LRU/LFU) and write strategies.
ratelimit.py	40	Fixed-window **rate limiter** (Redis INCR or in-memory).

---

ingestion/ — the live GitHub layer

File	~Lines	Purpose
github_client.py	241	**Async GitHub GraphQL client** — fetch repos/commits/PRs/issues within rate limits, guarded by the circuit breaker.
sync.py	141	**Ingestion core** — shared by the API (await) and Celery (background); turns raw activity into evidence and writes the graph.
tasks.py	148	**Celery task definitions** for ingestion + intelligence; eager fallback.
resume_parser.py	54	Extract text + canonical skills from a resume (PDF/docx); optional.

`intelligence/` — evidence reasoning

File	~Lines	Purpose
`confidence_scorer.py`	90	**Confidence formula** — source-weighted, complexity-scaled, recency-decayed trust per skill claim.
`skill_extractor.py`	107	Map free-text/messy skill strings to **canonical** names (spaCy optional, taxonomy fallback).
`summariser.py`	171	Turn a raw artifact (commit diff, PR review, issue, SO answer) into a short summary (Gemini optional, heuristic fallback).

`graph/` — the proof-of-work knowledge graph

File	~Lines	Purpose
`schema.py`	176	Graph schema + an **in-memory backend** (the fallback when Neo4j is off); `all_evidence()` feeds the vector index.
`queries.py`	230	**`GraphWriter`** / **`GraphQuerier`** over a pluggable backend (Neo4j or in-memory); confidence accumulates across evidence.
`vector_index.py`	130	**Semantic index** over evidence summaries (embeddings + cosine) for hybrid retrieval; reuses `make_embedder`, cached per evidence count.
`natural_language_query.py`	165	Recruiter English → **hybrid** retrieval: structured skill traversal + semantic vector search, fused with RRF; results tagged `match_via`.
`evaluate.py`	90	Graph-RAG **eval harness** — precision@k, recall@k, MRR, hit@k, `compare()` + `lift()` against a gold set.

`frontend/` — React recruiter UI

File	Purpose
`src/main.tsx`, `src/App.tsx`	App bootstrap + routes.
`src/pages/Login.tsx`	Login screen (JWT auth).
`src/pages/Dashboard.tsx`	Run a ranking, see job status.
`src/pages/RankingResults.tsx`	The ranked shortlist + reasons.
`src/pages/NaturalLanguageSearch.tsx`	Query the evidence graph in English.
`src/pages/Settings.tsx`	API keys, account settings.
`src/components/Layout.tsx`, `components/ui.tsx`	Shell + reusable UI primitives.
`src/components/developer/EvidenceTimeline.tsx`, `ProofOfWorkCard.tsx`	Live-layer visualisations.
`src/hooks/useJobStream.ts`	Subscribes to the SSE job-progress stream.
`src/lib/api.ts`, `src/lib/auth.tsx`	API client + auth context.
`vite.config.ts`, `tsconfig*.json`, `package.json`, `Dockerfile`	Build/config.

Infrastructure & config

File	Purpose
`docker-compose.yml`	**13 services** — postgres, redis, precompute, api2 (replica), neo4j, worker-ingestion, worker-intelligence, beat, gateway (nginx), frontend, prometheus, grafana, jaeger + volumes.
`Dockerfile`, `frontend/Dockerfile`	Multi-stage images for the API and the UI.
`.github/workflows/ci.yml`	CI: install, lint (Ruff), run the test suite.
`deploy/prometheus/*`, `deploy/grafana/*`	Prometheus scrape config + Grafana dashboards/datasource provisioning.
`requirements.txt`	All Python dependencies (every heavy one is optional at runtime).
`pyproject.toml`, `pytest.ini`, `alembic.ini`	Tooling config (Ruff, pytest, Alembic).
`scripts/make_dev_sample.py`	Build a small **stratified** sample dataset for fast local runs.
`submission_metadata*.yaml`	Hackathon submission metadata.

`tests/` — the safety net (68 tests)

File	Covers
`conftest.py`	Shared fixtures + a **candidate factory** that emits schema-valid profiles (and specific edge cases: honeypots, stuffers).
`test_feature_engineering.py`	The 40-feature builder.
`test_honeypot_detector.py`	Honeypot rules (precision).
`test_gate_filter.py`	Hard gates.
`test_accuracy_layer.py`	Eval metrics, graded pseudo-labels, reranker, tournament, scorer head-blend.
`test_performance.py`	Device selection, focused rerank text, training downsample, the content cache.
`test_reasoning_generator.py`, `test_submission_format.py`	Output + CSV validity.
`test_rank_smoke.py`	End-to-end: the live path yields a valid submission, fast.
`test_api.py`	API integration against a temp SQLite DB + in-memory cache.
`test_queue.py`	Queue/worker + cache eviction.
`test_live_layer.py`	Confidence formula, summariser, skill extraction, graph.
`test_graph_rag.py`	Hybrid graph+vector retrieval vs skill-only on a gold set (recall/MRR lift), eval metrics, vector index.

Next: 05_build_from_scratch.md to put it all together yourself.

05 — Build it from scratch

A milestone-by-milestone roadmap for recreating ProvenanceRank yourself. Each milestone produces something you can *run and verify* before moving on. You don't need the heavy libraries to start — build the pure-Python core first, then layer the smart bits on top.

> **Mindset:** build the simplest thing that produces a valid `submission.csv`, verify it, then improve accuracy, then wrap it in a service. Never let a fancy dependency become a hard requirement — always have a fallback.

Prerequisites

- **Python 3.11+**. Create a virtual environment: `python -m venv .venv && source .venv/bin/activate`.
- Start with just `numpy`, `pandas`, `joblib`. Add the rest later.
- (For the UI, much later) **Node 18+**.
- Get the dataset (`candidates.jsonl`) and a `job_description.txt`.

Milestone 0 — Project skeleton

Create the folders from 01_overview.md's repo map and a `core/config.py` that reads settings from environment variables with sensible defaults (paths, weights, model names). Add `core/logging.py` for JSON logs.

Why first: one place for configuration and logging means every later module stays clean. **Verify:** `python -c "from core.config import get_settings; print(get_settings().top_k)"` prints `100`.

Milestone 1 — Load data and build features

1. `pipeline/loader.py`: a generator that yields one JSON profile per line, plus a `corpus_text()` that flattens a profile into one string.
2. `core/constants.py`: define `NUMERIC_FEATURE_COLUMNS` — the list of ~40 feature names. This is your **contract**: every model agrees on these columns.
3. `pipeline/feature_engineering.py`: `build_feature_row(candidate) -> dict` that computes those 40 numbers (experience, tenure, skill matches, assessment scores, behavioural composite, location fit…), and `frame_from_rows()` to stack them into a pandas DataFrame.

Verify: stream 100 candidates, build the frame, print `df.shape` → `(100, ~40+)`. Check there are no NaNs in the numeric columns.

Milestone 2 — Honeypots and gates

1. `pipeline/honeypot_detector.py`: a handful of **strict** rules that flag impossible profiles (e.g. summed role durations exceed a plausible career length). Optimise for *precision* — never flag a real person.
2. `pipeline/gate_filter.py`: `apply_gates(df, spec)` that marks candidates failing hard requirements (consulting-only, keyword-stuffer, inactive). Keep them in the frame but flagged, and write the reasons to `excluded_candidates.csv`.

Verify: on the real data you should see roughly ~65 honeypots flagged and tens of thousands gated — and zero honeypots in your final top 100.

Milestone 3 — Retrieval (find the relevant candidates)

1. `pipeline/bm25_indexer.py`: build a BM25 keyword index over the profile texts. Start with the `rank-bm25` library; later add a streaming inverted-index fallback if memory is tight.
2. `pipeline/embedder.py`: encode profiles into vectors. Start with the **hashing embedder** (pure NumPy — no torch needed): hash each token to a bucket and sign, sum, L2-normalise. Add `sentence-transformers` later as the high-quality path.
3. `pipeline/retrieval.py`: `cosine_scores()` for the dense side and `fuse_rrf()` to combine BM25 ranks and embedding ranks via Reciprocal Rank Fusion.

Verify: for the JD, print the top 10 by `retrieval_score`. They should look topically relevant (titles/skills matching the JD).

Milestone 4 — The two entry points and a first submission

1. `pipeline/scorer.py`: `merge_final_scores()` — start simple: `0.40·ml_fit + 0.35·retrieval + 0.25·behavioural`, then multiply by location and availability factors, and force honeypots/gated to 0.
2. `ml/trainer.py` + `ml/predictor.py`: with no labels, build **proxy labels** from features you trust and fit a gradient-boosted regressor (or just return the proxy as a formula). `predict_fit()` loads the model, or falls back to the formula.
3. `output/reasoning_generator.py`: a template that writes one sentence per pick.
4. `output/submission_writer.py`: write exactly 100 rows and run the validator.
5. `precompute.py`: stream once → features → gates → embeddings → BM25 → retrieval → train → save artifacts.
6. `rank.py`: load artifacts → predict → merge → drop honeypots → top 100 → reasons → write + validate.

Verify: `python precompute.py …` then `python rank.py …` then `python validate_submission.py submission.csv` → **“Submission is valid.”** You now have a working ranker. Everything after this is making it *better* and *bigger*.

Milestone 5 — Measure, then improve accuracy

1. `ml/evaluate.py`: implement `NDCG@k`, `MAP`, `P@k`, and the weighted `composite`. Without this you’re guessing.
2. `ml/pseudo_labels.py`: build **graded** labels (tiers 0–5) by bucketing a rich composite by percentile. (Optional: have Gemini grade a sample.)
3. Upgrade `ml/trainer.py` with `train_ranker()` — an `XGBRanker` with `objective="rank:ndcg"` (LambdaMART) trained on those graded labels.
4. `ml/reranker.py`: a cross-encoder that re-scores the top ~300 (with a lexical fallback). Cache `rerank_score`.
5. `pipeline/tournament.py`: a Bradley-Terry pairwise tournament over the top ~60. Cache `tournament_score`.
6. Fold both into `scorer.py` as a convex head-blend, and wire all of it into `precompute.py`.

Verify: use the eval harness to A/B baseline vs. the accuracy layer on the graded labels — the composite should go up and the top-20 should reorder.

Milestone 6 — Make the offline phase fast

1. `core/device.py`: auto-detect MPS/CUDA; optional ONNX/int8 toggle.

2. `core/artifact_cache.py`: hash each stage's inputs; reuse the artifact on a re-run when the key matches. Wire it into `precompute.py` so changing only the JD reuses the embeddings.
3. In `trainer.py`: train on all positives + sampled negatives, histogram trees, early stopping.

Verify: a second `precompute --resume` with unchanged inputs finishes in ~1s; changing only the JD reuses the embeddings (look for `cache.hit` in the logs).

Milestone 7 — Wrap it in a production API

1. `db/` — SQLAlchemy 2.0 async models + a repository layer + Alembic migrations. Use SQLite for dev (it's a fallback for Postgres).
2. `core/security.py` + `api/routers/auth.py` — bcrypt passwords, JWTs, API keys.
3. `services/ranker.py` — load the engine once, keep it warm.
4. `services/queue.py` + `services/jobs.py` — a job queue + worker; the request returns a `job_id`, progress streams over SSE.
5. `services/cache.py` + `services/ratelimit.py` — Redis with in-memory fallback.
6. `api/` — the FastAPI app, routers, schemas, middleware, error handlers.
7. `core/logging.py`, `api/metrics.py`, `core/tracing.py`, `core/circuit_breaker.py` — observability + resilience.

Verify: `uvicorn api.main:app` → open `/docs`, log in as the bootstrap admin, submit a ranking job, watch it stream to completion.

Milestone 8 — The live signal ingestion layer

1. `intelligence/confidence_scorer.py` — the source-weighted, recency-decayed trust formula. (Build this first; it's the heart of the layer.)
2. `intelligence/skill_extractor.py` + `summariser.py` — canonicalise skills, summarise artifacts (both with heuristic fallbacks).
3. `ingestion/github_client.py` + `sync.py` + `tasks.py` — pull GitHub activity (async, rate-limited, circuit-breaker-guarded), turn it into evidence.
4. `graph/schema.py` + `queries.py` + `natural_language_query.py` — a SHA-256-anchored proof-of-work graph (Neo4j or in-memory) and an English-to-query endpoint.
5. Compose it back: `feature_engineering.apply_live_signals()` adds an additive `evidence_confidence_bonus` (default 0) that `scorer.py` includes — so the graded submission is unchanged when the layer is off.

Verify: without a GitHub token everything no-ops cleanly; with one, ingest a developer and query "who has shipped production LLM systems?" in plain English.

Milestone 9 — Frontend, Docker, CI

1. `frontend/` — a React + TypeScript + Vite app (Tailwind, React Query, router): Login, Dashboard, Ranking Results, NL Search, Settings; an SSE hook for live progress.
2. `Dockerfile` + `docker-compose.yml` — multi-stage images and the full stack (Postgres, Redis, Neo4j, API replicas, nginx gateway, workers, Prometheus, Grafana, Jaeger, the UI).
3. `.github/workflows/ci.yml` — install, lint with Ruff, run pytest.

Verify: `docker compose up --build` brings the whole system up; the UI talks to the API through the nginx gateway.

The order, in one line

features → honeypots/gates → retrieval → first valid submission → measure → accuracy layer → speed → API → live layer → UI/Docker/CI.

Build each milestone with its fallback path first; add the heavy library as an upgrade, never a requirement. That single rule is what lets the finished project run anywhere from a bare sandbox to a full GPU cluster.

See 06_glossary.md for any unfamiliar term.

06 — Glossary

Plain-English definitions of every technical term used in this project. Grouped by theme; skim for whatever you need.

Ranking quality metrics

- **Relevance / graded relevance** — how good a candidate is for the JD. Here it's graded on tiers 0–5 (5 = ideal hire, 0 = irrelevant), not just yes/no.
- **NDCG@k (Normalised Discounted Cumulative Gain)** — the main quality metric. It rewards putting highly-relevant people near the top of the first *k* results. "Discounted" = lower positions count less; "Normalised" = divided by the best possible ordering, so it lands in [0, 1]. `NDCG@10` looks at the top 10.
- **MAP (Mean Average Precision)** — averages how early the relevant candidates appear across the list. Rewards clustering good matches near the top.
- **P@10 (Precision at 10)** — the fraction of the top 10 that are actually relevant (here, tier ≥ 3).
- **Precision@k / Recall@k** — precision@k = of the k you returned, how many were relevant; recall@k = of all relevant items, how many you surfaced in the top k. Used to evaluate the graph search. Raising recall often lowers precision (you return more, catching more good ones but also more noise) — the **precision/recall trade-off**.
- **MRR (Mean Reciprocal Rank)** — 1 / the rank of the first relevant result, averaged over queries. Rewards putting a good answer at the very top.
- **Composite score** — the contest's weighted blend: `0.50·NDCG@10 + 0.30·NDCG@50 + 0.15·MAP + 0.05·P@10`. The number you're optimising.

Search & retrieval

- **BM25 (Okapi BM25)** — the standard keyword-relevance formula. Scores a document by how many query terms it contains, weighting **rarer** terms more and damping very long documents. "Sparse" retrieval (works on exact words).
- **TF-IDF / IDF** — term-frequency × inverse-document-frequency. The idea BM25 builds on: common words count for little, rare words count for a lot.
- **Inverted index** — a map from each word to the list of documents containing it. The data structure that makes keyword search fast.
- **Embedding** — a list of numbers (a vector) representing the *meaning* of text, so that similar meanings sit close together. Here, 384 numbers per profile.
- **Cosine similarity** — measures the angle between two vectors; ~1 = very similar, ~0 = unrelated. How we compare an embedding to the JD's embedding. "Dense" retrieval (works on meaning, not exact words).
- **Bi-encoder** — encodes the query and each document *separately* into vectors, then compares. Fast (you can pre-compute all document vectors); used for first-stage retrieval. sentence-transformers' default mode.
- **Cross-encoder** — feeds the query and a document *together* through the model and outputs one relevance score. Much more accurate, much slower — so we only run it on the top ~300 (the **reranker**).
- **RRF (Reciprocal Rank Fusion)** — a simple way to merge two ranked lists: add up `1/(k + rank)` from each list. Combines BM25 and embeddings without needing their scores to be on the same scale.
- **Hybrid retrieval** — using keyword (sparse) and semantic (dense) search together, then fusing them. More robust than either alone.

Machine learning

- **Feature / feature vector** — the fixed list of numbers describing one candidate (here, 40). Models only ever see these numbers.

- **GBDT (Gradient-Boosted Decision Trees)** — an ML model that combines many small decision trees, each correcting the last. XGBoost, LightGBM, and scikit-learn's HistGradientBoosting are all GBDTs. Great for tabular/feature data.
- **Learning-to-rank (LTR)** — training a model to *order* items, not to predict a number in isolation. Three flavours: **pointwise** (score each item), **pairwise** (which of two is better), **listwise** (optimise the whole list's metric).
- **LambdaMART** — a listwise LTR method: GBDTs trained to directly improve a ranking metric like NDCG. We use it via XGBoost's `XGBRanker(objective="rank:ndcg")`.
- **Pseudo-labels** — labels you *create* when the dataset has none. We build graded relevance tiers from a trusted composite (and optionally an LLM) so the LTR model has something to learn from.
- **Proxy label / formula** — a hand-written relevance estimate from features we trust. Used to bootstrap training and as the ultimate fallback if no ML library is installed.
- **Bradley-Terry model** — a classic model for "who beats whom" data. Given the results of many pairwise matches, it estimates each player's **strength** so the strengths best explain the results — even resolving cycles (A>B>C>A). We fit it with **MM iteration** (a stable, pure-NumPy maximum-likelihood algorithm).
- **MLE (Maximum Likelihood Estimation)** — pick the parameters that make the observed data most probable. How the Bradley-Terry strengths are found.
- **Normalisation (min-max)** — rescaling numbers into [0, 1] so different signals can be combined fairly.

The two-phase design & performance

- **Two-phase split** — slow, smart work runs **offline** (`precompute.py`) and is cached; the **online** graded step (`rank.py`) only reads the cache, so it meets the no-network/CPU/<5-min limits.
- **Graceful degradation / fallback** — if a heavy dependency is missing, the code uses a simpler pure-Python path instead of crashing. The project's core design rule.
- **Content-hash cache** — store an artifact with a hash of its inputs; on a re-run, reuse it only if the hash still matches. Lets a JD change reuse the embeddings.
- **MPS / CUDA** — Apple-Silicon GPU / NVIDIA GPU. The transformers auto-detect and use them when present.
- **ONNX / quantization (int8)** — ONNX Runtime is a fast model-execution engine; quantization shrinks model weights to 8-bit integers for ~2–4× CPU speed with tiny quality loss.

Production service

- **FastAPI** — the Python web framework serving the HTTP API.
- **ORM (Object-Relational Mapping)** — write Python objects, the library (SQLAlchemy) turns them into SQL. `db/models.py` are ORM models.
- **Migration (Alembic)** — a versioned change to the database schema, so the DB can evolve safely over time.
- **Repository pattern** — all database access goes through one layer (`db/repositories.py`), so the rest of the code never writes raw SQL.
- **Strategy pattern** — pick one of several interchangeable implementations at runtime (e.g. XGBoost → LightGBM → sklearn → formula). Keeps fallbacks clean.
- **JWT (JSON Web Token)** — a signed token proving who you are, sent on each request after login.
- **API key** — a long-lived secret a machine/script uses to authenticate (instead of logging in). Stored hashed; shown to you once.
- **RBAC (Role-Based Access Control)** — permissions by role (admin / recruiter / viewer).
- **bcrypt** — a deliberately slow password-hashing algorithm, so stolen hashes are hard to crack.
- **Redis** — a fast in-memory data store used here for caching and rate limiting.
- **Cache eviction (LRU / LFU)** — when the cache is full, drop the **L**east **R**ecently / **F**requently **U**sed entries.
- **Rate limiting (fixed window)** — cap requests per time window to prevent abuse.
- **Message queue** — a buffer between "a request arrived" and "the work ran", so the API can respond instantly and a worker processes jobs in the background.

- **Idempotent** — running the same operation twice has the same effect as once (so a duplicate job message is safely ignored).
- **At-least-once delivery** — the queue guarantees a message is processed at least once (retrying on failure); idempotency makes the possible duplicates harmless.
- **Dead-letter queue (DLQ)** — where messages go after too many failed retries, so they don't loop forever.
- **SSE (Server-Sent Events)** — a one-way stream from server to browser; used to push live job progress to the UI.
- **Circuit breaker** — stops calling a failing dependency for a while after repeated errors, then tests if it has recovered. Prevents cascading failure.
- **Prometheus / Grafana** — Prometheus collects metrics; Grafana charts them.
- **OpenTelemetry / Jaeger** — OpenTelemetry produces traces (the path of a request through the system); Jaeger displays them.
- **structlog** — logging library that emits structured JSON (machine-readable logs).
- **Celery** — runs background tasks on worker processes; "eager" mode runs them inline so no broker is needed in dev.

The live signal ingestion layer

- **Proof-of-work graph** — a knowledge graph linking developers → skills → evidence (commits, PRs, issues). Each evidence node is **SHA-256 anchored** (content-hashed) so it's tamper-evident/verifiable.
- **Knowledge graph / Neo4j** — data stored as nodes and relationships; Neo4j is the graph database (with an in-memory fallback here).
- **Confidence score** — how much to trust a skill claim, from the evidence: source-weighted (a merged PR > a resume line), complexity-scaled, and **recency-decayed**.
- **Recency decay / half-life** — older evidence counts for less; a one-year half-life means evidence is worth half as much after a year.
- **Evidence bonus** — the additive term (default 0) by which graph-verified developers float slightly above identical static-only profiles. It augments the ranking, never overrides it.
- **GraphQL** — the query language GitHub's API uses; lets us fetch exactly the fields we need in one request.

Project-specific terms

- **Honeypot** — a deliberately fake profile in the dataset (impossible career maths) planted as a trap; detected by strict rules and forced to score 0.
- **Gate / gating** — hard disqualifiers checked before ranking (e.g. consulting-only career, keyword stuffing, inactivity).
- **JobSpec** — the structured version of the job description (required skills, seniority, location) produced by the JD deconstructor.
- **Head (of the ranking)** — the top slice of candidates (~300 for rerank, ~60 for the tournament) where the score is actually decided.
- **Manifest** — `manifest.json`, the record of what a precompute run produced (hashes, counts, timings) used by `--resume`.

Back to the index.

07 — Running guide (every section, one by one)

How to actually run each part of ProvenanceRank — the ranking engine, the tests, the accuracy + performance layers, the API, the frontend, the live GitHub layer, the graph RAG search, the full Docker stack, and observability. Copy-paste commands throughout.

There are three "levels" and you can stop at whichever you need:

- **Level 1 — the engine only** (Sections 1–5): produces the graded `submission.csv`. Needs only Python.
- **Level 2 — the app, no Docker** (Sections 6–9): the API + UI on SQLite + in-memory fallbacks. Needs Python + Node.
- **Level 3 — the full stack** (Section 10): Postgres, Redis, Neo4j, workers, nginx, Prometheus, Grafana, Jaeger, the UI. Needs Docker.

> **Shortcut:** a `Makefile` wraps the common commands. Run `make help` to list them. Each section below notes the `make` target where one exists.

Quick start — Docker Desktop (the fewest steps)

Recommended on a laptop: the lite stack, from a clean state

The lite stack is 6 containers instead of 14 — far less disk/RAM and far fewer ways to fail. If a previous run got into a bad state, start clean:

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank

# 1. dataset must be a real FILE at the repo root (one-time)
[ -f candidates.jsonl ] || unzip -p "../[PUB] India_runs_data_and_ai_challenge.zip" \
  "*/candidates.jsonl" > candidates.jsonl

# 2. reclaim Docker disk + clear any half-built volumes from earlier attempts
docker system prune -af
docker compose -f docker-compose.lite.yml down -v

# 3. (optional) live-layer tokens — set BEFORE 'up'
export GITHUB_TOKEN=ghp_xxx
export GOOGLE_API_KEY=AIza_xxx

# 4. bring it up (first run builds images + the feature matrix; ~a few minutes)
docker compose -f docker-compose.lite.yml up --build

# 5. wait until the ranker is ready, THEN open the UI:
curl -s localhost:8080/health/ready          # ok = ready
#   open http://localhost:3001  (log in: admin@provenancerank.local / changeme-admin)
```

Stop it with `docker compose -f docker-compose.lite.yml down`.

> Don't click **Run ranking** until `feature_matrix.pkl` exists and `/health/ready` is ok — otherwise it ranks 0 candidates and fails. Check: `docker compose -f docker-compose.lite.yml exec api1 ls -lh /app/artifacts | grep feature_matrix`

Full stack (all 14 services)

If Docker Desktop is running and you want the complete stack (Neo4j, workers, Prometheus/Grafana/Jaeger), run these **in order**, once:

```
# 1. go to the repo
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
```

```
# 2. make sure the dataset is a real FILE at the repo root (one-time).
#    If you see a "candidates.jsonl" *directory*, remove it first: rm -rf candidates.jsonl
[ -f candidates.jsonl ] || unzip -p "../[PUB] India_runs_data_and_ai_challenge.zip" \
  "*/candidates.jsonl" > candidates.jsonl

# 3. (optional) tokens for the live layer — set BEFORE 'up'
export GITHUB_TOKEN=ghp_xxx        # GitHub ingestion
export GOOGLE_API_KEY=AIza_xxx     # Gemini JD parse + summaries

# 4. bring up the entire stack (first run builds images; later runs are fast)
docker compose up --build          # or: make up
```

Then just open:

URL	What
http://localhost:3001	recruiter UI
http://localhost:8080/docs	API (via the nginx gateway)
http://localhost:7474	Neo4j browser (neo4j / provenancerank)
http://localhost:3000	Grafana · http://localhost:16686 Jaeger · :9090 Prometheus

You do **not** run `precompute`, `uvicorn`, or `npm` yourself — the stack runs the `precompute` container automatically and starts all 13 services. Stop with `docker compose down` (add `-v` to wipe data). Full details + the lean-image caveat are in **Section 10**.

> First boot builds images and runs precompute, so give it a few minutes. The precompute step uses `--resume`, so on later ups it reuses the cached artifacts (stored in a Docker volume) and skips the rebuild.

---

0. One-time setup

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt          # or: make install
```

Put the dataset at the repo root (it's large, so it's gitignored):

```
cp /path/to/candidates.jsonl ./candidates.jsonl
ls -la candidates.jsonl     # must be a FILE (not a directory), ~480 MB
```

Every heavy dependency is optional — without them the code uses a pure-Python fallback (see 02_architecture.md, the degradation ladder). For a fast, lean install that skips torch/Gemini:

```
grep -viE "sentence-transformers|google-generativeai" requirements.txt > req.lean.txt
pip install -r req.lean.txt
```

---

1. The ranking engine → `submission.csv` (the graded core)

Two phases. **Phase 1** (offline, slow OK) builds artifacts; **Phase 2** (the graded step: no network, CPU, < 5 min) reads them.

```
# Phase 1 — build features, embeddings, BM25, the model, rerank + tournament
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt
```

```
#   make precompute

# Phase 2 — the graded step
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
#   make rank

# verify against the official validator
python validate_submission.py submission.csv      # -> "Submission is valid."
```

Useful `precompute.py` flags:

Flag	What it does
`--no-st`	skip sentence-transformers, use the hashing embedder (≈30s on 100K)
`--bm25-backend inverted`	memory-frugal keyword index (pairs well with `--no-st`)
`--no-accuracy`	skip the rerank/tournament layer (original proxy regressor)
`--resume`	reuse cached artifacts whose inputs are unchanged
`--limit N`	only process the first N candidates (dev only)

Fast offline run: `python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.t --no-st --bm25-backend inverted`.

Do I need to re-run precompute every time?

No. Precompute is a one-time build per *(dataset, JD)*. It writes everything to `artifacts/`; after that, run `rank.py` as many times as you like — it just reads the cache and finishes in ~2 seconds. You only re-run `precompute` when an **input changes**:

What changed	Re-run precompute?	Cost (with `--resume`)
Nothing	No — just run `rank.py`	—
The job description only	Yes, with `--resume`	seconds — reuses the 100K embeddings + the trained model, only redoes retrieval/rerank/tournament
The candidate dataset	Yes, with `--resume`	rebuilds (inputs changed)
Engine code / config	Yes	rebuilds the affected parts

Always add `--resume` after the first build — it hashes the inputs and recomputes only what actually changed (and skips entirely, ~1s, if nothing did). So your slow 8-minute embedding step is paid **once**:

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume
python rank.py --candidates ./candidates.jsonl --out ./submission.csv     # repeat freely
```

---

2. Tests & lint

```
python -m pytest -q          # the whole suite (68 tests)   |  make test
python -m pytest tests/test_accuracy_layer.py -q            # one file
python -m pytest -k tournament -q                           # by keyword

python -m ruff check .       # lint                          |  make lint
python -m ruff check . --fix # auto-fix
```

---

3. The accuracy layer (measure the lift)

The layer (graded-label LambdaMART + cross-encoder rerank + Bradley-Terry tournament) runs automatically inside `precompute.py`. To **prove** it helps, A/B it with the eval harness (`ml/evaluate.py`):

```
# baseline (layer off)
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --no-acc
python rank.py --candidates ./candidates.jsonl --out ./submission_base.csv

# full (layer on)
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt
python rank.py --candidates ./candidates.jsonl --out ./submission_full.csv
```

To compute NDCG@10/@50, MAP, P@10 yourself, import the harness:

```
from ml.evaluate import evaluate
metrics = evaluate(ranked_ids, relevance)   # relevance = {candidate_id: grade}
print(metrics["composite"])
```

Full detail: `../ACCURACY.md`.

---

4. Performance levers (speed, no feature loss)

All controlled by environment variables (or `core/config.py`); see `../ACCURACY.md` "Keeping it fast".

```
export COMPUTE_DEVICE=auto    # auto | cpu | cuda | mps  (GPU auto-detected)
export USE_ONNX=true          # ONNX Runtime + int8 (needs optimum[onnxruntime])
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume
```

`--resume` + the content-hash cache means a second run with unchanged inputs is near-instant, and changing only the JD reuses the 100K embeddings (look for `cache.hit` in the logs).

---

5. (Optional) build a small sample dataset

For quick iteration without the full 100K:

```
python scripts/make_dev_sample.py     # writes a stratified data/dev_sample.jsonl
python precompute.py --candidates ./data/dev_sample.jsonl --jd ./data/job_description.txt --n
```

---

6. The API (no Docker)

The API falls back to **SQLite + in-memory cache/graph/queue**, so nothing else needs to be installed.

```
source .venv/bin/activate
uvicorn api.main:app --reload --port 8000      #  make serve
# open http://localhost:8000/docs  (interactive Swagger UI for every endpoint)
```

A **bootstrap admin** is created on first boot:

- email: `admin@provenancerank.local`
- password: `changeme-admin`

Auth flow (curl)

```
# 1) log in -> get a JWT
TOKEN=$(curl -s -X POST localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@provenancerank.local","password":"changeme-admin"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# 2) call an authed endpoint
curl -s localhost:8000/auth/me -H "Authorization: Bearer $TOKEN"

# 3) (optional) mint an API key for scripts/machines
curl -s -X POST localhost:8000/auth/api-keys -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"name":"my-script"}'
# then send it as:  -H "X-API-Key: <the key shown once>"
```

Endpoint map

Method & path	What
`GET /health`, `/health/live`, `/health/ready`	liveness / readiness
`POST /auth/login`, `/auth/register`, `GET /auth/me`	auth
`POST /auth/api-keys`, `GET /auth/api-keys`	API keys
`POST /rank`	submit a ranking job → `job_id`
`GET /rank`, `GET /rank/{job_id}`	list / status
`GET /rank/{job_id}/results`	the ranked top-100
`GET /rank/{job_id}/submission.csv`	download the CSV
`GET /rank/{job_id}/stream`	live progress (SSE)
`POST /developers/connect-github`	live layer — ingest a developer
`GET /developers/{id}/sync-status`	real sync status + counts
`GET /developers/{id}/evidence-graph`	the proof-of-work subgraph
`POST /search/natural-language`	graph RAG search (English)
`GET /search/by-skill/{skill}`	structured skill lookup
`GET /admin/artifacts`, `POST /admin/precompute`, `GET /admin/jobs`, `/admin/audit`	admin
`GET /metrics`	Prometheus metrics

Run a ranking via the API

```
curl -s -X POST localhost:8000/rank -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"top_k":100}'
# -> {"job_id":"..."}  ; then poll:
curl -s localhost:8000/rank/<job_id> -H "Authorization: Bearer $TOKEN"
```

(Needs the Phase-1 artifacts from Section 1 to exist.)

---

7. The frontend (recruiter UI, no Docker)

```
cd frontend
npm install
npm run dev
# open http://localhost:5173
```

Log in with the bootstrap admin above. Pages: **Dashboard** (run a ranking), **Rankings** (results), **Evidence Search** (the live/graph layer), **Settings** (API keys). The dev server proxies API calls to `localhost:8000`, so run the API (Section 6) alongside it.

---

8. The live GitHub layer (evidence ingestion)

This is the differentiator; the core ranking does **not** need it. To ingest real activity you must give the **server process** a GitHub token:

```
export GITHUB_TOKEN=ghp_your_personal_access_token   # classic PAT; read-only OK
export GOOGLE_API_KEY=AIza_xxx                        # optional: Gemini summaries
uvicorn api.main:app --reload --port 8000             # restart so it sees the token
# verify the server actually sees it:
python -c "from core.config import get_settings; print('token seen:', bool(get_settings().git
```

Then ingest and check the **real** status (don't trust the UI's "Syncing…" label):

```
# connect a developer
curl -s -X POST localhost:8000/developers/connect-github -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"github_username":"octocat"}'
# -> {"developer_id":"...","status":"syncing"}

# the truth: synced / error + how many artifacts were ingested
curl -s localhost:8000/developers/<developer_id>/sync-status -H "Authorization: Bearer $TOKEN

# the proof-of-work subgraph
curl -s localhost:8000/developers/<developer_id>/evidence-graph -H "Authorization: Bearer $TO
```

Without a token, ingestion fails fast and the developer goes to `error` (the server log shows `developer.sync_failed`). Background work runs **eager** (inline) unless you run the Celery workers (Section 10).

---

9. Graph RAG / natural-language search

Ask in English; the system fuses exact skill traversal with semantic vector search over evidence summaries (see 03_features.md E4).

```
curl -s -X POST localhost:8000/search/natural-language -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"query":"who built a recommendation system at scale"}'
```

Each result is tagged `match_via` (`skill` / `semantic` / `both`). Flags (env vars): `GRAPH_HYBRID_ENABLED` (default true), `GRAPH_VECTOR_TOP_K`, `GRAPH_RRF_K`, `NEO4J_ENABLED` (false = in-memory graph, still fully functional).

Measure precision/recall with the graph eval harness:

```
python -m pytest tests/test_graph_rag.py -q     # baseline vs hybrid, asserts the lift
# or import graph.evaluate (precision@k, recall@k, MRR, compare, lift)
```

---

10. The full production stack (Docker)

Needs **Docker Desktop**. Brings up 13 services.

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
# candidates.jsonl must be a real FILE at the repo root (see Section 0) —
# if it's missing, Docker creates an empty *directory* and precompute fails.
docker compose up --build           #  make up   /   make down to stop
```

Set tokens for the workers (in your shell, an `.env`, or the compose env):

```
export GITHUB_TOKEN=ghp_xxx
export GOOGLE_API_KEY=AIza_xxx
```

Where everything lives

URL	Service
http://localhost:3001	recruiter UI (frontend)
http://localhost:8080/docs	API via the nginx gateway
http://localhost:7474	Neo4j browser (`neo4j` / `provenancerank`)
http://localhost:9090	Prometheus
http://localhost:3000	Grafana (anonymous)
http://localhost:16686	Jaeger traces

Boot order is handled for you: `precompute` runs once to build artifacts, then the API replicas, workers, and beat start. `docker compose down -v` also wipes the data volumes.

10b. Lite stack (6 containers instead of 14)

On a smaller machine the full 14-container stack is heavy. `docker-compose.lite.yml` runs the **same app** with far fewer moving parts — great for demos and laptops:

```
docker compose -f docker-compose.lite.yml up --build
# open http://localhost:3001 (UI)  and  http://localhost:8080/docs (API)
docker compose -f docker-compose.lite.yml down        # stop it
```

It keeps Postgres, Redis, precompute, one API, the nginx gateway, and the UI. It drops **neo4j** (graph falls back to the in-memory store), the **Celery workers + beat** (ingestion runs inside the API; only the auto-scheduled re-sync is gone), and **Prometheus/Grafana/Jaeger** (the API still emits metrics/logs/traces — you just don't run the dashboards). **No product feature is lost** and the graded `submission.csv` is unaffected. Switch back to the full stack any time with the normal `docker compose up`.

---

11. Observability

What	How to run / view
Metrics	`GET /metrics` on the API; scraped by Prometheus (`:9090`)
Dashboards	Grafana at `:3000` (provisioned from `deploy/grafana/`)
Tracing	enable `OTEL_ENABLED=true`; view spans in Jaeger (`:16686`)
Logs	structured JSON to stdout; `make logs` tails the API container

---

12. Database & migrations

Dev uses SQLite automatically. For Postgres (prod), apply migrations:

```
alembic upgrade head        #  make migrate
export DATABASE_URL="postgresql+asyncpg://user:pass@localhost:5432/provenancerank"
```

`make clean` removes caches + the local SQLite DB to start fresh.

LLM provider — GLM first, local Ollama fallback

All four LLM touchpoints — JD parsing, evidence summaries, pseudo-label grading, and natural-language query parsing — go through one helper (`core/llm.py`). In the default `auto` mode it tries, in order: **GLM** (your Z.ai/Zhipu key) → **local Ollama** → **Gemini** → the deterministic heuristic. So your GLM key is used first, and if a call fails or the key is unset it falls back to a local model.

```
# primary: GLM (Z.ai / Zhipu)
export GLM_API_KEY=your_glm_key
export GLM_MODEL=glm-5.2                  # default

# fallback: local Ollama (used if GLM is unset or a call fails)
brew install ollama && ollama serve
ollama pull llama3.1:8b                   # or qwen3-coder:30b on a 24GB+ Mac

python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume
```

Force a single provider with `LLM_PROVIDER=glm` (or `ollama` / `gemini` / `none`). GLM is called over its OpenAI-compatible endpoint, so `GLM_BASE_URL` can point at any compatible API.

Use the LLM to improve the *ranking* (`LLM_HEAD_SCORING_ENABLED`)

Beyond JD parsing, the LLM can directly score the **top of the candidate list**. Enabled, precompute has the LLM read the top ~250 candidates against the JD, score fit 0–100, cache it as `llm_fit_score`, and the scorer blends it into the head (weight `LLM_SCORE_WEIGHT`, default 0.15). `rank.py` only reads the cached column, so it stays no-network/<5min.

```
export GLM_API_KEY=your_glm_key
export LLM_HEAD_SCORING_ENABLED=true        # off by default
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt

# one-command A/B: builds both rankings from the same matrix, prints the metric
# delta, and writes submission_base.csv + submission_llm.csv to eyeball the names
python scripts/ab_llm.py
```

`ab_llm.py` scores with-vs-without the LLM column against the graded pseudo-labels and prints `composite` / `ndcg@10` lift + head churn — so you only keep it if it actually helps. It costs ~a dozen LLM calls offline; with no LLM reachable the whole thing is a silent no-op.

In Docker, Ollama runs on the host, so point the containers at it:

```
export OLLAMA_BASE_URL=http://host.docker.internal:11434
export LLM_PROVIDER=ollama OLLAMA_MODEL=llama3.1:8b
docker compose -f docker-compose.lite.yml up --build
```

> The LLM steps are still optional. `precompute`/`rank` produce a valid `submission.csv` with no LLM at all (rule-based JD parse + heuristic labels).

Environment-variable quick reference

Variable	Default	Controls
`GITHUB_TOKEN`	—	live-layer GitHub ingestion (required for it)
`LLM_PROVIDER`	`auto`	which LLM to use: `auto` (GLM → Ollama → Gemini) / `glm` / `ollama` / `gemini` / `none`
`GLM_API_KEY`	—	Z.ai / Zhipu **GLM** key — used **first** in `auto` when set
`GLM_MODEL`	`glm-5.2`	GLM model name
`GLM_BASE_URL`	`https://api.z.ai/api/paas/v4`	any OpenAI-compatible endpoint
`OLLAMA_BASE_URL`	`http://localhost:11434`	local Ollama fallback (Docker: `http://host.docker.internal:11434`)
`OLLAMA_MODEL`	`llama3.1:8b`	any model you've `ollama pull`-ed
`GOOGLE_API_KEY`	—	Gemini (last resort if neither GLM nor Ollama is available)
`PSEUDO_LABEL_LLM_ENABLED`	`false`	opt in to LLM *label grading* in precompute (slower; off by default)

Variable	Default	Controls
`LLM_HEAD_SCORING_ENABLED`	`false`	opt in to LLM *fit-scoring* the top ~250 (blended into the ranking)
`LLM_SCORE_WEIGHT`	`0.15`	blend weight for `llm_fit_score` in the head
`COMPUTE_DEVICE`	`auto`	transformer device: auto / cpu / cuda / mps
`USE_ONNX`	`false`	ONNX Runtime + int8 backend
`NEO4J_ENABLED`	`false`	use Neo4j (else in-memory graph)
`GRAPH_HYBRID_ENABLED`	`true`	hybrid graph+vector search
`DATABASE_URL`	sqlite	DB connection (Postgres in prod)
`REDIS_URL`	redis://...	cache + rate-limit + queue (falls back in-memory)
`CELERY_TASK_ALWAYS_EAGER`	`true`	run background tasks inline (no worker)
`OTEL_ENABLED`	`false`	OpenTelemetry tracing
`JWT_SECRET`	dev value	**change in production**
`BOOTSTRAP_ADMIN_EMAIL/PASSWORD`	admin@... / changeme-admin	first admin user

(Anything in `core/config.py` can be set this way — the env var is the upper-cased field name.)

Troubleshooting

Symptom	Cause & fix
`IsADirectoryError: '/app/candidates.jsonl'` (Docker)	No dataset file at the repo root → Docker made an empty dir. `docker compose down`, `rm -rf candidates.jsonl`, copy the real file, up again.
UI stuck on “Syncing...”	Cosmetic — the page doesn’t poll. Check the **real** status at `/developers/{id}/sync-status` or the server log (`sync.complete` vs `developer.sync_failed`).
`GITHUB_TOKEN not configured` in logs	The server process can’t see the token. Export it in the **same shell** that runs uvicorn (or the compose env) and restart. Verify with the one-liner in Section 8.
`rank.py` errors “feature matrix missing”	Run `precompute.py` first (Section 1).
torch install slow/large	Use the lean install (Section 0) + `precompute --no-st`.
precompute “freezes” after `bm25.build`	With a `GOOGLE_API_KEY` set, LLM label-grading used to run 75 slow Gemini calls here. It’s now **off by default** (heuristic labels, instant). Opt in with `PSEUDO_LABEL_LLM_ENABLED=true` if you want it.
Port already in use	Change `--port` (API) or Vite’s port in `frontend/vite.config.ts`.
beat crashes: `Permission denied: 'celerybeat-schedule'`	`/app` is root-owned but the container runs as app. Fixed in compose (writes the schedule to `/tmp`). Pull the latest `docker-compose.yml`.
`neo4j` exits (3) “again and again”	Usually a *cascade*: another container (e.g. `beat`) crashes, aborts `docker compose up`, and kills neo4j mid-startup; plus a stale data volume from earlier aborted runs. Fix: `docker compose down -v` then `docker compose up --build`. See the real reason with `docker compose logs neo4j`.
Reset local DB	`rm provenancerank.db*` (or `make clean`).

`make` targets at a glance

```
make install     # pip install -r requirements.txt
make precompute  # build offline artifacts
make rank        # produce submission.csv
make serve       # run the API (sqlite + in-memory)
make test        # run the test suite
make lint        # ruff + mypy
```

```
make migrate     # apply DB migrations
make up / down   # full Docker stack up / down
make logs        # tail API logs
make clean       # remove caches + local db
```

That’s every section. Start at **Section 1** for the graded output; everything else layers on top without changing it.